# TP3 — Deep Learning : classification automatique de produits e-commerce

**Matière :** Les fondamentaux de l'IA **Niveau :** Bachelor 3 **Durée :** 1h30–2h **Objectifs pédagogiques :** - Comprendre la différence entre ML classique et Deep Learning sur un cas industriel concret - Construire et entraîner un réseau de neurones dense puis convolutif (CNN) - Analyser les courbes d'apprentissage pour diagnostiquer overfitting / underfitting - Comparer les performances ML vs DL et argumenter le choix d'architecture pour la production

## Contexte entreprise

Vous êtes **Data Scientist junior** dans l'équipe Catalog Intelligence de **Zalando**, la plus grande plateforme e-commerce de mode en Europe (50 millions de clients actifs, 45 millions d'articles).

**Problème métier :** Sur la marketplace, les vendeurs tiers uploadent des photos de produits et saisissent manuellement la catégorie (T-shirt, pantalon, robe, etc.). En production, **12 % des articles sont mal catégorisés**, ce qui dégrade l'expérience de recherche et coûte **2,3 M EUR/an** en retours liés à des erreurs de catalogue.

**Votre mission :** Développer un système de **classification automatique des images produits** pour détecter et corriger les erreurs de catégorisation avant publication. L'objectif business est de passer le taux d'erreur de 12 % à moins de 5 %.

**Dataset :** Le dataset **Fashion-MNIST** que vous allez utiliser est un jeu de données publié par l'équipe Research de Zalando en 2017. Il contient **70 000 images réelles** extraites du catalogue Zalando, converties en niveaux de gris 28×28 pixels et réparties en 10 catégories de vêtements/accessoires. Ce sont de **véritables photos de produits** issues de la plateforme.

**Source :** Han Xiao, Kashif Rasul, Roland Vollgraf — *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms* (Zalando Research, 2017).

## Prérequis

```bash
pip install tensorflow scikit-learn matplotlib numpy seaborn
```

Ou directement sur **Google Colab** (TensorFlow préinstallé) : colab.research.google.com

## Étape 1 — Charger et explorer les données produits Zalando

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras

# Chargement du dataset Zalando Fashion-MNIST (téléchargement automatique ~30 Mo)
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()

# Les 10 catégories du catalogue Zalando
class_names = [
    'T-shirt/top', 'Pantalon', 'Pull', 'Robe', 'Manteau',
    'Sandale', 'Chemise', 'Sneaker', 'Sac', 'Bottine'
]

print("=== EXPLORATION DU CATALOGUE ===")
print(f"Train : {X_train.shape[0]} images de {X_train.shape[1]}x{X_train.shape[2]} pixels")
print(f"Test  : {X_test.shape[0]} images")
print(f"Pixels : min={X_train.min()}, max={X_train.max()} (niveaux de gris 0-255)")
print(f"Catégories : {len(class_names)}")

# Distribution des catégories
print("\nDistribution des catégories (train) :")
unique, counts = np.unique(y_train, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  {class_names[u]:12s} : {c:>5d} articles ({c/len(y_train)*100:.1f}%)")
```

### Visualisation : exemples du catalogue

```python
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    idx = np.where(y_train == i)[0][0]
    ax.imshow(X_train[idx], cmap='gray')
    ax.set_title(class_names[i], fontsize=10)
    ax.axis('off')
plt.suptitle('Catalogue Zalando — 1 exemple par catégorie', fontsize=13)
plt.tight_layout()
plt.savefig('catalogue_samples.png')
plt.show()
```

**Questions :** 1. Le catalogue est-il équilibré entre les catégories ? Quel impact sur l'entraînement ? 2. Un pull et une chemise se ressemblent visuellement : cela pose-t-il un risque de confusion pour un algorithme ? 3. La résolution 28×28 pixels est-elle représentative d'un cas de production réel ? Quelle résolution utiliserait-on sur la vraie plateforme Zalando ?

## Étape 2 — Prétraitement des images

```python
# Normalisation : [0, 255] → [0.0, 1.0]
# Les réseaux de neurones convergent mieux avec des valeurs entre 0 et 1
X_train_norm = X_train.astype('float32') / 255.0
X_test_norm  = X_test.astype('float32') / 255.0

print(f"Après normalisation : min={X_train_norm.min():.1f}, max={X_train_norm.max():.1f}")

# Pour scikit-learn : aplatir chaque image 28x28 en vecteur de 784 valeurs
X_train_flat = X_train_norm.reshape(-1, 784)
X_test_flat  = X_test_norm.reshape(-1, 784)

print(f"Forme pour ML classique (aplatie)  : {X_train_flat.shape}")
print(f"Forme pour réseau dense (grille)   : {X_train_norm.shape}")
```

**Question :** Pourquoi normalise-t-on les pixels entre 0 et 1 avant d'entraîner un réseau de neurones ? Que se passerait-il si on gardait les valeurs brutes 0–255 ?

## Étape 3 — Baseline : Random Forest (ML classique)

Avant de déployer un modèle Deep Learning en production, l'équipe exige une **baseline** avec un modèle classique pour justifier la complexité supplémentaire.

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Random Forest sur les pixels aplatis (784 features = 784 pixels individuels)
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train_flat, y_train)

y_pred_rf = rf.predict(X_test_flat)
acc_rf = accuracy_score(y_test, y_pred_rf)
print(f"Random Forest — Accuracy : {acc_rf*100:.1f}%")
```

**Questions :** 1. Cette accuracy suffit-elle pour atteindre l'objectif business (taux d'erreur < 5 %) ? 2. Random Forest traite chaque pixel comme une feature indépendante : comprend-il que les pixels voisins forment un contour ou une texture ?

## Étape 4 — Réseau de neurones dense (MLP)

Un réseau dense (Multi-Layer Perceptron) connecte chaque neurone de la couche précédente à chaque neurone de la couche suivante. Comme le Random Forest, il ne tient pas compte de la position spatiale des pixels dans l'image.

```python
# Architecture : Input(28x28) → Flatten → Dense(128) → Dense(64) → Dense(10)
model_dense = keras.Sequential([
    keras.layers.Input(shape=(28, 28)),
    keras.layers.Flatten(),                      # 28x28 → 784
    keras.layers.Dense(128, activation='relu'),   # Couche cachée 1
    keras.layers.Dense(64, activation='relu'),    # Couche cachée 2
    keras.layers.Dense(10, activation='softmax')  # 10 catégories → probabilités
])

model_dense.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',  # Classification multi-classe avec labels entiers
    metrics=['accuracy']
)

model_dense.summary()

# Entraînement — 15% du train réservé pour la validation
history_dense = model_dense.fit(
    X_train_norm, y_train,
    epochs=15,
    batch_size=64,
    validation_split=0.15,
    verbose=1
)
```

### Comprendre l'architecture

| COUCHE | RÔLE | PARAMÈTRES |
|---|---|---|
| `Flatten` | Transforme l'image 28x28 en vecteur de 784 valeurs | 0 |
| `Dense(128, relu)` | Couche cachée : détecte des combinaisons de pixels | 784x128 + 128 = 100 480 |
| `Dense(64, relu)` | Couche cachée : combine les motifs détectés | 128x64 + 64 = 8 256 |
| **`Dense(10, softmax)`** | **Sortie : probabilité pour chaque catégorie** | **64x10 + 10 = 650** |

**Questions :** 1. Combien de paramètres entraînables au total ? Comparez avec le nombre de pixels d'une image (784). 2. Que fait `softmax` sur la dernière couche ? Pourquoi est-ce adapté à une classification multi-classe ? 3. Pourquoi `sparse_categorical_crossentropy` et pas `binary_crossentropy` ?

---

## Étape 5 — Réseau convolutif (CNN)

Un CNN exploite la **structure spatiale** des images. Au lieu de traiter chaque pixel indépendamment, il applique des petits filtres (3x3 pixels) qui glissent sur l'image pour détecter des motifs locaux : contours, textures, puis formes de plus en plus complexes.

C'est exactement ce que fait un opérateur humain du contrôle qualité : il repère des formes (manches, col, semelle) pour décider de la catégorie.

PYTHON

```python
# Ajouter la dimension canal (niveaux de gris = 1 canal, couleur RGB = 3 canaux)
X_train_cnn = X_train_norm.reshape(-1, 28, 28, 1)
X_test_cnn  = X_test_norm.reshape(-1, 28, 28, 1)

print(f"Forme pour CNN : {X_train_cnn.shape}")  # (60000, 28, 28, 1)

model_cnn = keras.Sequential([
    keras.layers.Input(shape=(28, 28, 1)),

    # Bloc 1 : détection de motifs simples (contours, bords)
    keras.layers.Conv2D(32, (3, 3), activation='relu'),   # 32 filtres 3x3
    keras.layers.MaxPooling2D((2, 2)),                     # Réduction spatiale

    # Bloc 2 : détection de motifs complexes (formes, structures)
    keras.layers.Conv2D(64, (3, 3), activation='relu'),   # 64 filtres 3x3
    keras.layers.MaxPooling2D((2, 2)),                     # Réduction spatiale

    # Classification finale
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.3),                             # Régularisation anti-overfitting
    keras.layers.Dense(10, activation='softmax')
])

model_cnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model_cnn.summary()

history_cnn = model_cnn.fit(
    X_train_cnn, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.15,
    verbose=1
)
```

### Comprendre le CNN couche par couche

| COUCHE | CE QU'ELLE FAIT | DIMENSION DE SORTIE |
|---|---|---|
| `Conv2D(32, 3x3)` | 32 filtres balaient l'image et détectent des motifs locaux | (26, 26, 32) |
| `MaxPooling2D(2x2)` | Garde le maximum par bloc 2x2, réduit la taille de moitié | (13, 13, 32) |
| `Conv2D(64, 3x3)` | 64 filtres combinent les motifs simples en motifs complexes | (11, 11, 64) |
| `MaxPooling2D(2x2)` | Nouvelle réduction spatiale | (5, 5, 64) |
| `Flatten` | Aplatit en vecteur pour la classification | 1 600 |
| `Dense(64) + Dropout` | Classification avec régularisation (30% de neurones désactivés) | 64 |
| **`Dense(10, softmax)`** | **Probabilité par catégorie** | **10** |

**Questions :** 1. Pourquoi la sortie de `Conv2D(32, 3x3)` appliquée à une image 28x28 est-elle 26x26 ? *(Indice : le filtre 3x3 ne peut pas déborder du bord.)* 2. Quel est l'intérêt du `MaxPooling2D` ? Que se passerait-il sans réduction spatiale ? 3. `Dropout(0.3)` désactive aléatoirement 30 % des neurones à chaque batch d'entraînement. Pourquoi cela réduit-il l'overfitting ?

## Étape 6 — Courbes d'apprentissage : diagnostic production

En production, un modèle qui surapprend (accuracy très haute en train, basse en validation) sera inutilisable sur les nouvelles photos uploadées par les vendeurs.

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Réseau dense
axes[0].plot(history_dense.history['accuracy'], 'b-', label='Train')
axes[0].plot(history_dense.history['val_accuracy'], 'r-', label='Validation')
axes[0].set_title('Réseau Dense (MLP)')
axes[0].set_xlabel('Époque')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# CNN
axes[1].plot(history_cnn.history['accuracy'], 'b-', label='Train')
axes[1].plot(history_cnn.history['val_accuracy'], 'r-', label='Validation')
axes[1].set_title('CNN')
axes[1].set_xlabel('Époque')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Courbes d\'apprentissage — Diagnostic overfitting', fontsize=13)
plt.tight_layout()
plt.savefig('learning_curves.png')
plt.show()
```

**Questions :** 1. L'écart train/validation indique-t-il de l'overfitting sur l'un des modèles ? 2. Le CNN converge-t-il plus vite ou plus lentement que le réseau dense ? 3. Si vous deviez mettre un modèle en production demain, quel nombre d'époques choisiriez-vous pour chaque modèle ?

## Étape 7 — Comparaison des 3 approches et décision

Le comité technique de Zalando exige un comparatif argumenté avant toute mise en production.

```python
from sklearn.metrics import classification_report, confusion_matrix

# Évaluation sur le jeu de test (données jamais vues)
loss_dense, acc_dense = model_dense.evaluate(X_test_norm, y_test, verbose=0)
loss_cnn, acc_cnn     = model_cnn.evaluate(X_test_cnn, y_test, verbose=0)

# Tableau comparatif
print("=" * 55)
print("  COMPARAISON DES 3 APPROCHES — COMITÉ TECHNIQUE")
print("=" * 55)
print(f"  {'Modèle':<20s} {'Accuracy':>10s} {'Taux erreur':>12s}")
print("-" * 55)
print(f"  {'Random Forest':<20s} {acc_rf*100:>9.1f}% {(1-acc_rf)*100:>11.1f}%")
print(f"  {'Réseau Dense':<20s} {acc_dense*100:>9.1f}% {(1-acc_dense)*100:>11.1f}%")
print(f"  {'CNN':<20s} {acc_cnn*100:>9.1f}% {(1-acc_cnn)*100:>11.1f}%")
print("-" * 55)
print(f"  Objectif business : taux d'erreur < 5.0%")
print("=" * 55)

# Prédictions du CNN pour l'analyse détaillée
y_pred_cnn     = model_cnn.predict(X_test_cnn, verbose=0)
y_pred_classes = np.argmax(y_pred_cnn, axis=1)

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Matrice de confusion — CNN (articles Zalando)')
plt.ylabel('Catégorie réelle')
plt.xlabel('Catégorie prédite')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('confusion_matrix_cnn.png')
plt.show()

# Rapport détaillé par catégorie
print("\n=== RAPPORT PAR CATÉGORIE — CNN ===")
print(classification_report(y_test, y_pred_classes, target_names=class_names))
```

**Questions :** 1. Quel modèle atteint l'objectif business (taux d'erreur < 5 %) ? 2. Quelles catégories sont le plus souvent confondues ? Est-ce cohérent (ex. : pull vs chemise) ? 3. Pour la mise en production, faut-il uniquement regarder l'accuracy globale, ou aussi le recall/precision par catégorie ? Pourquoi ?

## Étape 8 — Analyse des erreurs pour l'équipe produit

L'équipe Produit veut comprendre **quels types d'erreurs** le modèle commet pour décider s'il peut être déployé avec ou sans supervision humaine.

```python
# Articles mal classifiés par le CNN
errors = np.where(y_pred_classes != y_test)[0]
print(f"Erreurs : {len(errors)} / {len(y_test)} ({len(errors)/len(y_test)*100:.1f}%)")

# Afficher 10 erreurs types
fig, axes = plt.subplots(2, 5, figsize=(14, 6))
for i, ax in enumerate(axes.flat):
    idx = errors[i]
    ax.imshow(X_test[idx], cmap='gray')

    # Confiance du modèle sur la prédiction erronée
    confiance = y_pred_cnn[idx, y_pred_classes[idx]] * 100

    ax.set_title(
        f"Prédit : {class_names[y_pred_classes[idx]]} ({confiance:.0f}%)\n"
        f"Réel : {class_names[y_test[idx]]}",
        fontsize=8,
        color='red'
    )
    ax.axis('off')
plt.suptitle('CNN — Articles mal classifiés (analyse qualité)', fontsize=13)
plt.tight_layout()
plt.savefig('erreurs_cnn.png')
plt.show()

# Analyse : quelles paires de catégories posent le plus de problèmes ?
print("\n=== TOP 5 CONFUSIONS LES PLUS FRÉQUENTES ===")
confusions = {}
for real, pred in zip(y_test[errors], y_pred_classes[errors]):
    pair = (class_names[real], class_names[pred])
    confusions[pair] = confusions.get(pair, 0) + 1

top_confusions = sorted(confusions.items(), key=lambda x: x[1], reverse=True)[:5]
for (real, pred), count in top_confusions:
    print(f"  {real:12s} → classifié comme {pred:12s} : {count} erreurs")
```

**Questions :** 1. Les erreurs les plus fréquentes sont-elles compréhensibles visuellement (ex. : chemise vs T-shirt) ? 2. Le modèle fait-il des erreurs avec une **haute confiance** (> 90 %) ? Si oui, c'est un risque en production. Pourquoi ? 3. Pour les paires de catégories les plus confondues, proposez une solution business (ex. : fusionner les catégories, ajouter une validation humaine, enrichir les données).

## Étape 9 — Visualiser ce que le CNN a appris

Pour convaincre le comité éthique et l'équipe Produit que le modèle prend ses décisions sur des critères visuels pertinents (et non sur des artefacts), vous devez rendre le CNN **interprétable**.

### 9a — Filtres appris par la première couche

```python
# Extraire les 32 filtres 3x3 de la première couche convolutive
first_conv_layer = model_cnn.layers[0]
filters, biases = first_conv_layer.get_weights()
print(f"Filtres : {filters.shape}")  # (3, 3, 1, 32) — 32 filtres de 3x3 pixels

fig, axes = plt.subplots(4, 8, figsize=(12, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(filters[:, :, 0, i], cmap='gray')
    ax.set_title(f'F{i+1}', fontsize=7)
    ax.axis('off')
plt.suptitle('Filtres appris par le CNN — 1re couche Conv2D (3x3)', fontsize=13)
plt.tight_layout()
plt.savefig('filtres_conv.png')
plt.show()
```

### 9b — Ce que « voit » le CNN sur un article

```python
# Modèle intermédiaire qui renvoie les activations de la 1re couche Conv2D
activation_model = keras.Model(
    inputs=model_cnn.input,
    outputs=model_cnn.layers[0].output
)

# Choisir un article (Sneaker, catégorie 7)
sample_idx  = np.where(y_test == 7)[0][0]
sample      = X_test_cnn[sample_idx:sample_idx+1]
activations = activation_model.predict(sample, verbose=0)
print(f"Activations : {activations.shape}")  # (1, 26, 26, 32)

# Image originale + 8 feature maps
fig, axes = plt.subplots(1, 9, figsize=(16, 2.5))
axes[0].imshow(X_test[sample_idx], cmap='gray')
axes[0].set_title('Original', fontsize=9)
axes[0].axis('off')

for i in range(8):
    axes[i+1].imshow(activations[0, :, :, i], cmap='viridis')
    axes[i+1].set_title(f'Filtre {i+1}', fontsize=9)
    axes[i+1].axis('off')

plt.suptitle(f'Activations du CNN — {class_names[y_test[sample_idx]]}', fontsize=12)
plt.tight_layout()
plt.savefig('activations_conv.png')
plt.show()
```

**Questions :** 1. Les filtres appris détectent-ils des contours, des textures, ou des zones uniformes ? 2. Sur les feature maps, quels filtres réagissent à la silhouette du sneaker ? Lesquels réagissent au fond ? 3. Un comité éthique vous demande de prouver que le modèle ne discrimine pas selon la couleur de fond des photos. Ces visualisations suffisent-elles ? Que proposeriez-vous en complément ?

## Étape 10 — Debrief : recommandation au comité technique

Rédigez une synthèse structurée en **exactement 3 insights** pour le comité technique de Zalando :

**Insight 1 — ML classique vs Deep Learning** Observation : *(différence d'accuracy entre Random Forest et CNN, taux d'erreur atteint)* Explication : *(pourquoi le CNN surpasse Random Forest sur des images)* Recommandation : *(le surcoût GPU du CNN est-il justifié par le gain de performance ?)*

**Insight 2 — Dense vs CNN** Observation : *(écart de performance entre le réseau dense et le CNN)* Explication : *(quel mécanisme des convolutions explique cet avantage ?)* Recommandation : *(dans quel cas un réseau dense suffirait ?)*

**Insight 3 — Fiabilité en production** Observation : *(quelles catégories posent problème, confiance sur les erreurs)* Explication : *(pourquoi certaines confusions sont inévitables à cette résolution)* Recommandation : *(validation humaine sur les cas incertains, seuil de confiance ?)*

## Livrables

| LIVRABLE | DESCRIPTION |
|---|---|
| **Notebook propre/exécutable** | Toutes les étapes exécutées, cellules commentées, sans erreurs |
| **Tableau comparatif** | Accuracy et taux d'erreur des 3 modèles vs objectif business |
| **Courbes d'apprentissage** | Graphiques train vs validation avec diagnostic overfitting |
| **Matrice de confusion CNN** | Capture annotée, top 5 confusions identifiées |
| **Visualisation filtres/activations** | **Captures commentées (interprétabilité)** |

## Critères d'évaluation

| CRITÈRE | POINTS |
|---|---|
| Code fonctionnel et exécutable (étapes 1 à 9) | 4 |
| Comparaison 3 modèles : tableau et analyse argumentée | 4 |
| Courbes d'apprentissage : diagnostic overfitting commenté | 4 |
| Analyse des erreurs et matrice de confusion | 4 |
| 3 insights débrief : pertinence et justification business | 4 |
| **Total** | **20** |

## Ressources

- TensorFlow / Keras — Documentation officielle
- Fashion-MNIST — Zalando Research (GitHub)
- Google Colab
- CNN Explainer — Visualisation interactive
- scikit-learn — Documentation
- Dataset : Fashion-MNIST sur Kaggle (Zalando Research) — également intégré à `keras.datasets`